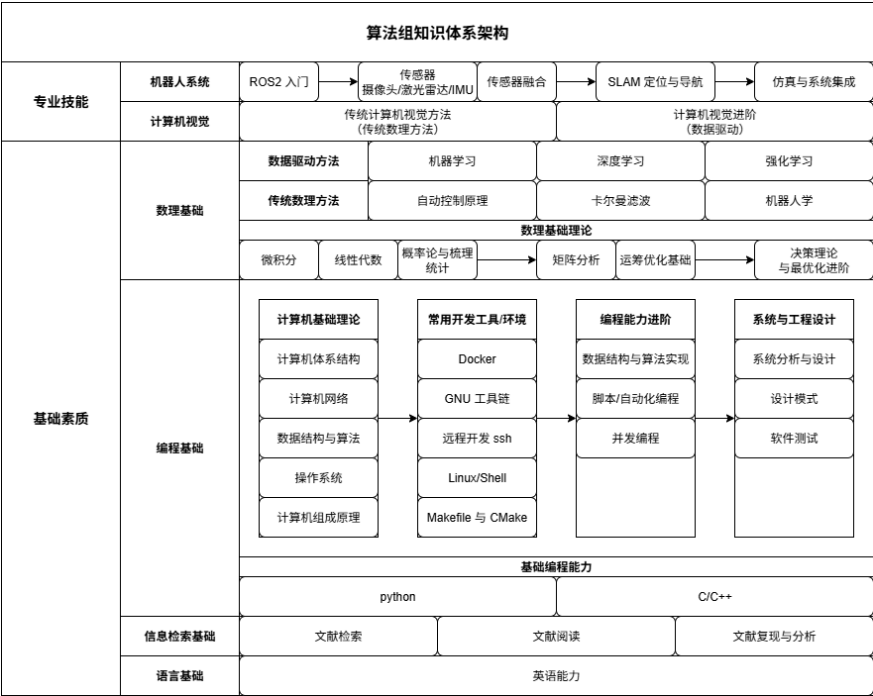


第一次培训

第一部分 知识体系介绍

算法组知识体系分为 **专业技能**、**数理基础**、**基础素质** 三大维度，覆盖从机器人系统到计算机视觉、从数学理论到编程实践的完整知识链。



一、专业技能维度

1. 计算机视觉方向

- **传统方法**：几何建模、图像处理、特征提取与匹配等。
- **数据驱动方法**：机器学习、深度学习、强化学习等。
- **核心目标**：理解视觉在 RoboMaster 中的应用场景，如装甲板识别与目标跟踪。

2. 机器人系统方向

- 学习路径：
ROS2 入门 → 传感器（相机/激光雷达/IMU） → 传感器融合 → SLAM 定位与导航 → 仿真与系统集成
- 核心目标：掌握机器人从感知到定位再到整体系统搭建的流程。

二、数理基础维度

1. 数理基础理论

- 微积分、线性代数、概率统计
- 进阶：矩阵分析、运筹优化、决策理论与最优化等
- 作用：为算法建模和推导提供数学工具。

2. 传统数理方法

- 自动控制原理、卡尔曼滤波、机器人学等
- 作用：支撑 SLAM、运动预测等算法的理论基础。

3. 数据驱动方法

- 机器学习、深度学习、强化学习等
- 作用：提供面向数据的算法设计思路。

三、基础素质维度

1. 编程基础

- 编程语言：Python、C/C++
- 计算机基础理论：体系结构、操作系统、计算机网络、数据结构与算法等
- 开发工具/环境：Linux/Shell、Git、CMake、Docker、SSH 等
- 能力进阶：并发编程、脚本自动化、设计模式、软件测试

- **作用：**保证能够高效开发和调试算法。

2. 信息检索基础

- 文献检索、文献阅读、文献复现与分析
- **作用：**提升学习和科研能力，能够快速追踪前沿算法。

3. 语言基础

- 英语能力
- **作用：**方便查阅国际前沿资料和开源文档。

第二部分 培训与招新流程

Part 1 编程语言入门

培训第一部分旨在为同学们打牢计算机编程基础，系统掌握 Python、C 语言与 C++ 的核心概念与编程技能。通过循序渐进的教学安排，同学们将从理解基本语法、控制流与数据结构开始，逐步过渡到函数、类及面向对象设计，再进一步掌握项目工程化与常用编程工具链的使用。在理论讲解的基础上，培训结合实践小任务，如基于 Python 的弹球射击小游戏和基于 C 的控制台 Flappy Bird 游戏，确保同学们能够将所学知识应用于实际开发场景，为后续计算机视觉及高级编程课程奠定坚实基础。

周次	Python	C语言	C++	备注
1	■			Python 环境 + 变量运算符
2	■			控制流、数据结构、函数、类、模块
3		■		CMD、GCC、变量、数组、指针
4		■		结构体、typedef、CMake 等
5		■		C语言实践任务
6			■	C++ 面向对象、引用与指针

周次	Python	C语言	C++	备注
7			■	C++ 继承、封装、多态
8			■	C++ 继承、封装、多态、标准容器

Python（2周）

- 环境配置
 - 安装 Python 及开发环境
- 基础知识
 - 变量与运算符、控制流（顺序、分支、循环）
 - 数据结构（列表、字典、集合等）
 - 函数、类（面向对象基础）
 - 模块（import 及自定义模块）
- 实践任务
 - 【待定】基于 pygame 编写弹球弹幕射击小游戏

C语言（3周）

- 环境与工具
 - CMD 命令行、GCC 工具链
- 基础知识
 - 变量、运算符、控制流、函数
 - 数组、指针、结构体与 typedef
- 项目管理
 - Makefile 与 CMake 及项目工程化
- 实践任务
 - 【待定】基于 C 编写控制台版 Flappy Bird 游戏

C++（3周）

- 面向对象
 - 类与对象、继承、封装、多态

- 引用与指针在类中的应用
- **数据结构**
 - C++ 标准容器（vector、map、set 等）
- **进阶（自学，可选）**
 - 运算符重载
 - 模板元编程

Part 2 计算机视觉入门

第二部分培训旨在引导同学们初步掌握计算机视觉的基本原理与实践技能，重点围绕 OpenCV 工具库的应用展开。同学们将在 Python 和 C++ 环境中，系统学习图像的基本处理方法，包括色彩空间转换、阈值处理、形态学操作以及常用图形学操作等内容。通过理论讲解与动手实践相结合的方式，同学们能够理解图像处理的核心概念，并在编程环境中实现基本的视觉算法，为后续深度视觉任务及机器人感知模块的开发打下坚实基础。

OpenCV 入门（1个月）

- 环境配置（python 与 C++）
- 色彩空间基础
- 基础形态学操作
- 阈值相关操作
- 常用图形学操作

【注】推荐大家通过 python 版本入门了解其基本使用方式，并同时熟悉 C++ 版本，入队考核要求使用 C++ 版本开发。

Part 3 入队考核

- **准备时间：**1 个月
- **考核内容：**
 - Python 与 C++ 基础知识
 - OpenCV 应用能力
- **考核方式：**
 - 一个基于 OpenCV 的计算机视觉小项目

第三部分 常用软件及网络资源介绍

这一部分培训主要介绍开发中常用的工具和资源，包括 IDE、Git/GitHub 和 Linux等。学员将掌握高效编程、版本管理、团队协作以及环境搭建的方法，同时了解常用在线资源和社区，为后续项目开发和实践打下基础。

AI 智能助手

- 基于人工智能和自然语言处理的工具，能够理解指令、回答问题、生成文本和代码。是我们算法组的“**大爹**”。
- 常见应用：
 - ChatGPT: <https://chatgpt.com>
 - GitHub Copilot: <https://github.com/features/copilot>
 - 豆包: <https://www.doubao.com/chat>
 - DeepSeek: <https://www.deepseek.com>
- AI 智能助手**不是取代人**，而是辅助工具。合理使用，可以大幅提升学习、开发和协作的效率。学习新语言、新算法时请**谨慎使用**。

IDE 与开发环境

- **IDE（集成开发环境）**
 - **VS Code**: 轻量级，插件生态丰富，适合 Python、C/C++ 开发、远程开发
 - **CLion / Visual Studio**: 更适合大型 C/C++ 项目
 - **PyCharm**: Python 专业开发环境
- **编辑器**
 - Vim / Neovim: 命令行轻量编辑器
 - Emacs: 高度可定制

Git 与版本控制

- **Git 基础**
 - 初始化仓库: `git init`

- 常用操作： `git add`、 `git commit`、 `git branch`、 `git merge`
- 分布式版本控制理念
- **进阶操作**
 - 远程仓库同步： `git push` / `git pull`
 - 分支协作与代码合并
 - 解决冲突的常见方法

GitHub 平台

- **代码托管与协作**
 - 创建和管理仓库
 - 使用 Issues 与 Pull Requests
 - 团队协作与开源贡献
- **GitHub Actions**
 - 持续集成（CI/CD）
 - 自动化构建与测试

Linux 基础

- **常见发行版**
 - Ubuntu / Debian（常用，社区活跃）
 - CentOS / Fedora（偏服务器）
- **常用命令**
 - 文件操作： `ls` , `cd` , `cp` , `mv` , `rm`
 - 系统管理： `ps` , `top` , `kill`
 - 软件安装： `apt` , `yum`
- **Shell 脚本**
 - 基础语法与自动化脚本

Markdown

- 一种轻量级标记语言
- 使用纯文本格式，快速生成格式化文档
- 常用于：

- 课程笔记
- 技术文档
- GitHub README
- 博客和论坛发布内容

利用学生身份白嫖资源

各类软件、云服务、硬件和学习平台有时针对在校学生提供的免费或折扣使用权益。大部分需要使用 [edu.com 邮箱](#) 进行申请，例如：

- [GitHub Student Developer Pack](#)
 - 申请方式可以参考：[Github学生认证及学生包保姆级申请指南](#)
- [JetBrains Student Pack](#)
 - 申请方式可以参考：[高校学生免费使用jetbrains产品指南（保姆级教程）](#)

其他优质资源

- Robomaster开源社区：<https://bbs.robomaster.com/>
- 优质博客：
 - oi-wiki：<https://oi-wiki.org/>
 - csdiy-wiki：<https://csdiy.wiki/>
- 图书馆资源
 - 合肥工业大学图书馆（校外请使用VPN登录）：<https://lib.hfut.edu.cn/portal/>
 - zlibrary：<https://zh.z-library.sk/>
- 示意图绘制：[visio](#)、[draw.io](#)、[graphviz](#)、[PlantUML](#)

什么是 VPN?

- VPN（Virtual Private Network，虚拟专用网络）
- 通过加密通道将设备安全地连接到远程网络
- 常用场景：
 - 远程办公访问公司内网
 - 科研或开发时**访问特定资源**
 - 保护公共网络中的隐私

- 实例：
 - [合肥工业大学 WebVPN](#)

课后小任务

必做任务：

- ☐ 下载并安装 Python。
- ☐ 配置一个 IDE（如 PyCharm、VS Code 等）。
- ☐ 注册 GitHub 账号。
- ☐ 下载并安装 Git。
- ☐ 创建一个新的本地项目，并在项目中添加一个 [README.md](#) 文件。
- ☐ 使用 Git 将项目提交到 GitHub。

拓展任务：

- ☐ 申请 GitHub Student Developer Pack，体验学生优惠资源。
- ☐ 尝试使用 Copilot。